

GenAI - Code is on the last page in a table

ChatGPT - Free version, no think mode

Gemini - Free version using the 3.6 Flash version that was defaulted

It is important to consider that turning on modes for each GenAI may have resulted in different results as it makes the AI think longer thus coming up with different solutions,

Both were accessed on their respective website i.e <https://chatgpt.com/> and <https://gemini.google.com/app>

Prompt:

Generate me a C program that will prioritize emails for a busy company CEO. The program will use a MaxHeap as a means of implementing a priority queue. A

priority queue is a queue where emails can shift towards the front of the queue

based on a priority status. The MaxHeap uses a list-based implementation, then use that MaxHeap to handle all the email prioritizing for the CEO. For the test file format,

Command	Description
EMAIL <sender category>,<subject line>,<date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

(Sent each GenAI this image)

You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Correctness

- In terms of correctness GPT although having more code in general is able to deal with more cases that weren't discussed including different line endings such as `\n` in addition to extra spaces after certain keywords that needed to be used like `Next` or `read`. In addition it also has an ordercounter that helps with the ordering. Gemini lacks all of these.

execution time

- However, since GPT has much more robust code, it also leads its execution time to be longer than Gemini. Gemini preferred to compare integers while GPT would rather format them into strings making the process several times longer for GPT.

Space Complexity

- They're both relatively the same, although GPT is slightly slower due to the arrays storing more memory in addition to some of the data that was more redundant than necessary.

Maintainability

- In general, GPT had way more comments more so over the code blocks themselves instead of every single line.

My Fixes: Assisted by AI

Decision: I will decide to use GPT for the source code.

Correctness

- One of the suggested ideas was to store the numeric dates inside the email structure rather than re-parsing it during each heap swaps

execution time

- A suggested idea from the genAI was to call `dataValue()` during insertion instead of calling it multiple times

Space Complexity

- It was suggested by the AI to reduce some of the arrays as two of them had excessive padding per node

Maintainability

- Replacing magic numbers to cleanly handle priority levels.

ChatGPT	Gemini
---------	--------

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_CAPACITY 10

// Structure for an email
typedef struct {
    char category[30];
    char subject[200];
    char date[11];    // MM-DD-YYYY
    int priority;
    int order;        // Used when category and date are the
same
} Email;

// Structure for the MaxHeap
typedef struct {
    Email *emails;
    int size;
    int capacity;
} MaxHeap;

// -----
// Return the priority value for each sender category
// Higher number = higher priority
// -----
int getPriority(char category[]) {
    if (strcmp(category, "Boss") == 0)
        return 5;
    else if (strcmp(category, "Subordinate") == 0)
        return 4;
    else if (strcmp(category, "Peer") == 0)
        return 3;
    else if (strcmp(category, "ImportantPerson") == 0)
        return 2;
    else
        return 1; // OtherPerson
}

// -----
// Convert MM-DD-YYYY into a number that can be
compared
// -----
int dateValue(char date[]) {
    int month, day, year;

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE 256
#define MAX_CATEGORY 32
#define MAX_SUBJECT 128
#define MAX_DATE 12

typedef struct {
    char sender[MAX_CATEGORY];
    char subject[MAX_SUBJECT];
    char date[MAX_DATE];
    int year;
    int month;
    int day;
    int priority_rank; // Higher rank means higher priority
} Email;

typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;

// Maps sender string to a numerical priority rank (Higher =
More Important)
int get_sender_rank(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    if (strcmp(sender, "OtherPerson") == 0) return 1;
    return 0;
}

// Parses "MM-DD-YYYY" into numeric components for
date comparison
void parse_date(const char *date_str, int *m, int *d, int *y) {
    sscanf(date_str, "%d-%d-%d", m, d, y);
}

// Compares two emails: returns >0 if e1 has higher priority
than e2
int compare_emails(Email e1, Email e2) {
    if (e1.priority_rank != e2.priority_rank) {
        return e1.priority_rank - e2.priority_rank;
    }
    // Tie-breaker: Newest date comes first
    if (e1.year != e2.year) return e1.year - e2.year;
}

```

```

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

// -----
// Determine whether email a has higher priority than email
// b
// -----
int higherPriority(Email a, Email b) {

    // First compare sender category
    if (a.priority > b.priority)
        return 1;

    if (a.priority < b.priority)
        return 0;

    // Same category: compare dates
    int dateA = dateValue(a.date);
    int dateB = dateValue(b.date);

    if (dateA > dateB)
        return 1;

    if (dateA < dateB)
        return 0;

    // Same category and date:
    // later email entered is considered newer
    if (a.order > b.order)
        return 1;

    return 0;
}

// -----
// Swap two emails
// -----
void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

// -----
// Initialize the MaxHeap

```

```

    if (e1.month != e2.month) return e1.month - e2.month;
    return e1.day - e2.day;
}

MaxHeap* create_heap(int initial_capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->data = (Email*)malloc(sizeof(Email) *
initial_capacity);
    heap->size = 0;
    heap->capacity = initial_capacity;
    return heap;
}

void swap_emails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(heap->data[index],
heap->data[parent]) > 0) {
            swap_emails(&heap->data[index],
&heap->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

void heapify_down(MaxHeap *heap, int index) {
    int max_idx = index;
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;

        if (left < heap->size &&
compare_emails(heap->data[left], heap->data[max_idx]) >
0) {
            max_idx = left;
        }
        if (right < heap->size &&
compare_emails(heap->data[right], heap->data[max_idx])
> 0) {
            max_idx = right;
        }
    }
}

```

```

// -----
void initializeHeap(MaxHeap *heap) {
    heap->emails = malloc(INITIAL_CAPACITY *
sizeof(Email));

    if (heap->emails == NULL) {
        printf("Memory allocation error.\n");
        exit(1);
    }

    heap->size = 0;
    heap->capacity = INITIAL_CAPACITY;
}

// -----
// Resize the heap when it becomes full
// -----
void resizeHeap(MaxHeap *heap) {
    heap->capacity *= 2;

    Email *temp = realloc(
        heap->emails,
        heap->capacity * sizeof(Email)
    );

    if (temp == NULL) {
        printf("Memory allocation error.\n");
        free(heap->emails);
        exit(1);
    }

    heap->emails = temp;
}

// -----
// Move an item upward in the MaxHeap
// -----
void heapifyUp(MaxHeap *heap, int index) {

    while (index > 0) {

        int parent = (index - 1) / 2;

        if (higherPriority(
            heap->emails[index],
            heap->emails[parent])) {

            swapEmails(

```

```

            if (max_idx != index) {
                swap_emails(&heap->data[index],
&heap->data[max_idx]);
                index = max_idx;
            } else {
                break;
            }
        }
    }

    void insert_email(MaxHeap *heap, Email email) {
        if (heap->size == heap->capacity) {
            heap->capacity *= 2;
            heap->data = (Email*)realloc(heap->data,
sizeof(Email) * heap->capacity);
        }
        heap->data[heap->size] = email;
        heapify_up(heap, heap->size);
        heap->size++;
    }

    // Removes the root (highest priority) element without
displaying
    void extract_max(MaxHeap *heap) {
        if (heap->size == 0) return;
        heap->data[0] = heap->data[heap->size - 1];
        heap->size--;
        if (heap->size > 0) {
            heapify_down(heap, 0);
        }
    }

    void free_heap(MaxHeap *heap) {
        free(heap->data);
        free(heap);
    }

    int main(int argc, char *argv[]) {
        FILE *file = stdin;
        if (argc > 1) {
            file = fopen(argv[1], "r");
            if (!file) {
                perror("Error opening file");
                return 1;
            }
        }

        MaxHeap *heap = create_heap(10);
        char line[MAX_LINE];

```

```

        &heap->emails[index],
        &heap->emails[parent]
    );

    index = parent;
}
else {
    break;
}
}
}

// -----
// Move an item downward in the MaxHeap
// -----
void heapifyDown(MaxHeap *heap, int index) {

    while (1) {

        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        // Check left child
        if (left < heap->size &&
            higherPriority(
                heap->emails[left],
                heap->emails[largest])) {

            largest = left;
        }

        // Check right child
        if (right < heap->size &&
            higherPriority(
                heap->emails[right],
                heap->emails[largest])) {

            largest = right;
        }

        // No child has higher priority
        if (largest == index)
            break;

        swapEmails(
            &heap->emails[index],
            &heap->emails[largest]
        );
    }
}

```

```

while (fgets(line, sizeof(line), file)) {
    // Remove trailing newline if present
    line[strcspn(line, "\r\n")] = 0;
    if (strlen(line) == 0) continue;

    if (strncmp(line, "EMAIL ", 6) == 0) {
        Email e;
        char *token_str = line + 6; // Move past "EMAIL "

        // Parse formatted input: <sender category>,
        // <subject line>, <date>
        char *sender = strtok(token_str, ",");
        char *subject = strtok(NULL, ",");
        char *date = strtok(NULL, ",");

        if (sender && subject && date) {
            // Trim leading whitespace on subject and date if
            // necessary
            while (*sender == ' ') sender++;
            while (*subject == ' ') subject++;
            while (*date == ' ') date++;

            strncpy(e.sender, sender, MAX_CATEGORY);
            strncpy(e.subject, subject, MAX_SUBJECT);
            strncpy(e.date, date, MAX_DATE);
            e.priority_rank = get_sender_rank(e.sender);
            parse_date(e.date, &e.month, &e.day, &e.year);

            insert_email(heap, e);
        }
        else if (strcmp(line, "NEXT") == 0) {
            if (heap->size == 0) {
                printf("No emails in queue.\n");
            }
            else {
                Email top = heap->data[0];
                printf("Sender: %s\nSubject: %s\nDate: %s\n",
                    top.sender, top.subject, top.date);
            }
        }
        else if (strcmp(line, "READ") == 0) {
            extract_max(heap);
        }
        else if (strcmp(line, "COUNT") == 0) {
            printf("Unread emails count: %d\n", heap->size);
        }
    }
}

if (file != stdin) fclose(file);
free_heap(heap);
return 0;
}

```

```

        index = largest;
    }
}

// -----
// Insert an email into the MaxHeap
// -----
void insertEmail(MaxHeap *heap, Email email) {

    if (heap->size == heap->capacity) {
        resizeHeap(heap);
    }

    // Put the new email at the end
    heap->emails[heap->size] = email;

    // Restore MaxHeap property
    heapifyUp(heap, heap->size);

    heap->size++;
}

// -----
// Return the highest-priority email without removing it
// -----
Email *peek(MaxHeap *heap) {

    if (heap->size == 0)
        return NULL;

    return &heap->emails[0];
}

// -----
// Remove the highest-priority email
// -----
void removeMax(MaxHeap *heap) {

    if (heap->size == 0)
        return;

    // Move last item to the root
    heap->emails[0] = heap->emails[heap->size - 1];

    heap->size--;
}

```

```

// Restore MaxHeap property
if (heap->size > 0) {
    heapifyDown(heap, 0);
}
}

// -----
// Remove leading/trailing whitespace
// -----
void trim(char *str) {

    // Remove leading spaces
    char *start = str;

    while (*start == ' ' || *start == '\t')
        start++;

    if (start != str) {
        memmove(str, start, strlen(start) + 1);
    }

    // Remove trailing spaces
    int length = strlen(str);

    while (length > 0 &&
        (str[length - 1] == ' ' ||
        str[length - 1] == '\t' ||
        str[length - 1] == '\n' ||
        str[length - 1] == '\r')) {

        str[length - 1] = '\0';
        length--;
    }
}

// -----
// Main program
// -----
int main(void) {

    MaxHeap heap;
    initializeHeap(&heap);

    char line[500];
    int orderCounter = 0;

    // Read commands until end of file
    while (fgets(line, sizeof(line), stdin) != NULL) {

```

```

trim(line);

// Ignore empty lines
if (strlen(line) == 0)
    continue;

// -----
// EMAIL command
// Format:
// EMAIL category,subject,date
// -----
if (strncmp(line, "EMAIL ", 6) == 0) {

    char *data = line + 6;

    char *category = strtok(data, ",");
    char *subject = strtok(NULL, ",");
    char *date = strtok(NULL, ",");

    if (category != NULL &&
        subject != NULL &&
        date != NULL) {

        trim(category);
        trim(subject);
        trim(date);

        Email email;

        strcpy(email.category, category);
        strcpy(email.subject, subject);
        strcpy(email.date, date);

        email.priority = getPriority(email.category);

        // Used to determine which email is newer
        email.order = orderCounter++;

        insertEmail(&heap, email);
    }
}

// -----
// NEXT command
// Display highest-priority email
// without removing it
// -----

```

```
else if (strcmp(line, "NEXT") == 0) {  
  
    Email *email = peek(&heap);  
  
    if (email != NULL) {  
  
        printf("Sender: %s\n", email->category);  
        printf("Subject: %s\n", email->subject);  
        printf("Date: %s\n", email->date);  
    }  
}
```

```
// -----  
// READ command  
// Remove highest-priority email  
// -----  
else if (strcmp(line, "READ") == 0) {  
  
    removeMax(&heap);  
}
```

```
// -----  
// COUNT command  
// Display number of unread emails  
// -----  
else if (strcmp(line, "COUNT") == 0) {  
  
    printf("%d\n", heap.size);  
}  
}
```

```
// Free dynamically allocated memory  
free(heap.emails);
```

```
return 0;
```

```
}
```